Algorithms

GREEDY ALGORITHMS

This Week

- Greedy algorithms
- Huffman Coding!

A general structure of a greedy algorithm and proof.

1. Preprocessing (like sort).
2. A sequence of local optimal choices.

Optimality proof:

Part 1: The greedy choice property:
If *S* is an instance whose local optimum is step 'a' then there exists an optimal solution that does step 'a'

Proof Method: Given a solution B that does not start with step a, show that either (i) B is not optimal, or (ii) [exchange argument]: B can be modified to do a, and the resulting solution is feasible and not worse than B.

A general structure of a greedy algorithm and proof.

Part 2: The optimality of the whole algorithm is by induction on the number of local choice iterations.

Induction Claim: For every k, there exists an optimal solution that agrees with the first k steps of the greedy algorithm.

Base case: k=0. An empty solution is a subset of the optimal solution.

Induction step: Based on the greedy choice property.

In particular, when the greedy algorithm terminates, there exists an optimal solution that is identical to the solution provided by the greedy algorithm.

Exercise 1: Job Assignment

We are given a set of n jobs, where the i-th job is associated with a processing time t_i.

A **schedule** assigns the jobs in some order on a single processor.

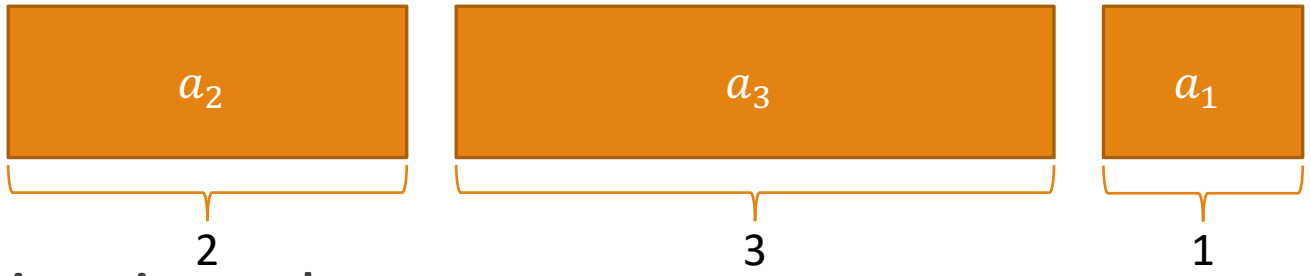
Denote by C_i the completion time of job i (the time it starts being processed + t_i).

The goal is to find a schedule in which the **total completion time** of the jobs is minimal.

Formally, the goal is to find a schedule such that $\sum_i C_i$ is minimal.

Provide a greedy algorithm for the problem, prove correctness and analyze time complexity.

Exercise 1: Job Assignment



Completion times in various orders:

$$a_1, a_2, a_3 \rightarrow 1 + 3 + 6 = 10$$
$$a_3, a_2, a_1 \rightarrow 3 + 5 + 6 = 14$$
$$a_2, a_3, a_1 \rightarrow 2 + 5 + 6 = 13$$

Algorithm: sort the jobs from the shortest to the longest and schedule them according to this order.

Time complexity: $O(n \cdot \log n)$.

Exercise 1: Job Assignment

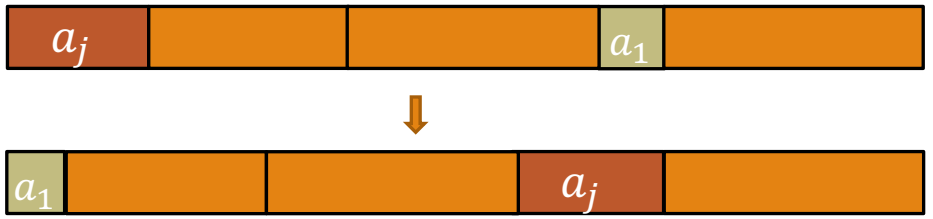
Greedy Choice Property: There exists an optimal schedule in which job a_1 (the shortest job) is first

Proof: Let B be a solution in which a_1 is not the first.

Let a_j be the first job in B. Let $\delta = t_j - t_1$. According to the sorting, we know that $\delta \geq 0$.

If we swap a_1 with a_j , we reduce the completion time of each job in locations $2, \ldots, j-1$ by δ, and do not hurt the completion times of the jobs in locations $j+1, \ldots, n$.

Also C_1+C_j reduces by δ. Thus, we can exchange a_j with a_1 without hurting the total completion time.



(exchange argument)

Exercise 1: Job Assignment

Claim: Let $A = (a_1, \ldots, a_n)$ be the output of the algorithm. For every $0 \leq k \leq n$ there exists an optimal solution B that agrees with A on the first k jobs.

Proof: For $k = 0$, trivial.

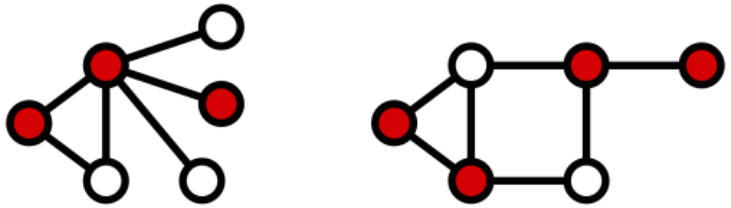
Let $k > 0$ and let B be an optimal solution such that $(a_1, \ldots, a_{k-1})$ are scheduled first. Let t be the completion time of job k-1. All the remaining jobs are assigned after job k-1 in both A and B.

Since a_k is the shortest not assigned job, from the GCP we know that there exists a solution for the remaining unassigned jobs in which a_k is first.

Therefore B' in which $(a_1, \ldots, a_{k-1}, a_k)$ are scheduled first is an optimal solution that agrees with A on the first k jobs.

Exercise 2: Vertex Cover

A *vertex cover* of an undirected graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ such that for every edge $(u, v) \in E$, it holds that $u \in S$ or $v \in S$.



Vertex Cover



Minimal Size Vertex Cover

Exercise 2: Vertex Cover

A *vertex cover* of an undirected graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ such that for every edge $(u, v) \in E$, it holds that $u \in S$ or $v \in S$.

Assume G is a **tree**. Suggest a linear time algorithm to find a vertex cover of minimal size.

Exercise 2: Vertex Cover

We notice that every leaf or its neighbor must be in the solution in order to obtain a valid vertex cover. We also notice it is better to choose the neighbor of the leaf, since it can cover additional edges.

Our algorithm will greedily pick a leaf and place the predecessor in the cover.

Algorithm 1 Min-Tree-Vertex-Cover

Set $S = \emptyset$
while E is not empty **do**
    Pick a leaf u
    Let v be the neighbor of u. Add v to S and remove all adjacent edges of v from G.
Return S

Exercise 2: Vertex Cover

Running time: Linear. Finding and maintaining an array with leaves can be done in linear time and every edge is removed exactly once (use postorder).

The greedy choice property: Let T be a tree, let u be a leaf in T and let v be the neighbor of u. Then, there exists a minimal size vertex cover that includes v.

Proof: Let W be an optimal solution. If $v \in W$, we are done.

Otherwise, since W is a valid vertex cover and $(u, v) \in E$, it must be that $u \in W$.

We replace u with v in W to obtain $W' = (W \cup \{v\}) \setminus \{u\}$. Since u is a leaf, the only edge which may be affected by this modification is (u, v), and since W' covers (u, v) as well, W' is a valid vertex cover. Last, $|W| = |W'|$, meaning W' is optimal as well.

Exercise 2: Vertex Cover

Claim: Let $S = \{v_1, \ldots, v_k\}$ $(k < n)$ be the vertex cover the algorithm returned. We prove by induction that for every $0 \leq i \leq k$, there exists an optimal vertex cover $W \subseteq V$ $s.t. \{v_1, \ldots, v_{i-1}\} \subseteq W$.

Proof: For $i = 0$, clearly $\emptyset \subseteq W$ for any optimal solution W.

Let $i > 0$ and let W be an optimal solution with $\{v_1, \ldots, v_{i-1}\} \subseteq W$. By the induction hypothesis, such an optimal solution exists.

Let T' be the subtree of T which is not covered by $\{v_1 \ldots v_{i-1}\}$. Note that T' is a forest and v_i is adjacent to a leaf in T' . Since $\{v_1, \ldots, v_{i-1}\}$ is an optimal solution for $T \setminus T'$, an optimal solution for T' together with $\{v_1, \ldots, v_{i-1}\}$ is an optimal solution for T.

By the greedy choice property, there exists an optimal solution W' for T' that includes v_i . Thus, $W = W' \cup \{v_1, \ldots, v_{i-1}\}$ is an optimal solution for T and $\{v_1, \ldots, v_i\} \subseteq W$.

For $i = k$ we get that S is optimal.

Huffman Coding

Input: A text over some alphabet C

Output: A prefix-free binary code of the alphabet such that the total length of the encoded text is minimal

Algorithm:

1. sort the alphabet in increasing order of frequencies
2. create a leaf for each character in the alphabet
3. while there is more than 1 character

3.1 combine the two least-frequent characters to a single character, by combining their nodes to a single node

4. position of a character in the tree is its decoding.

Complexity?

$O(|C|\log|C|)$

Exercise 3: Huffman coding

Construct a optimal Huffman tree for the following set of frequencies:

A	B	C	D	E
15	15	10	50	10

Exercise 3: Huffman coding

A; 15

B; 15

C; 10

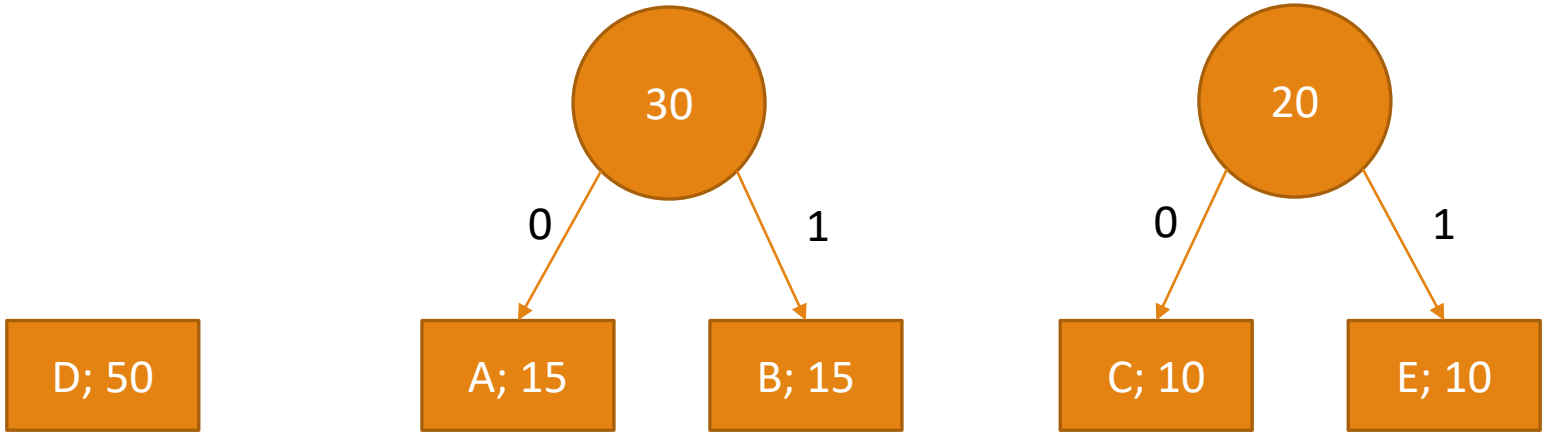
D; 50

E; 10

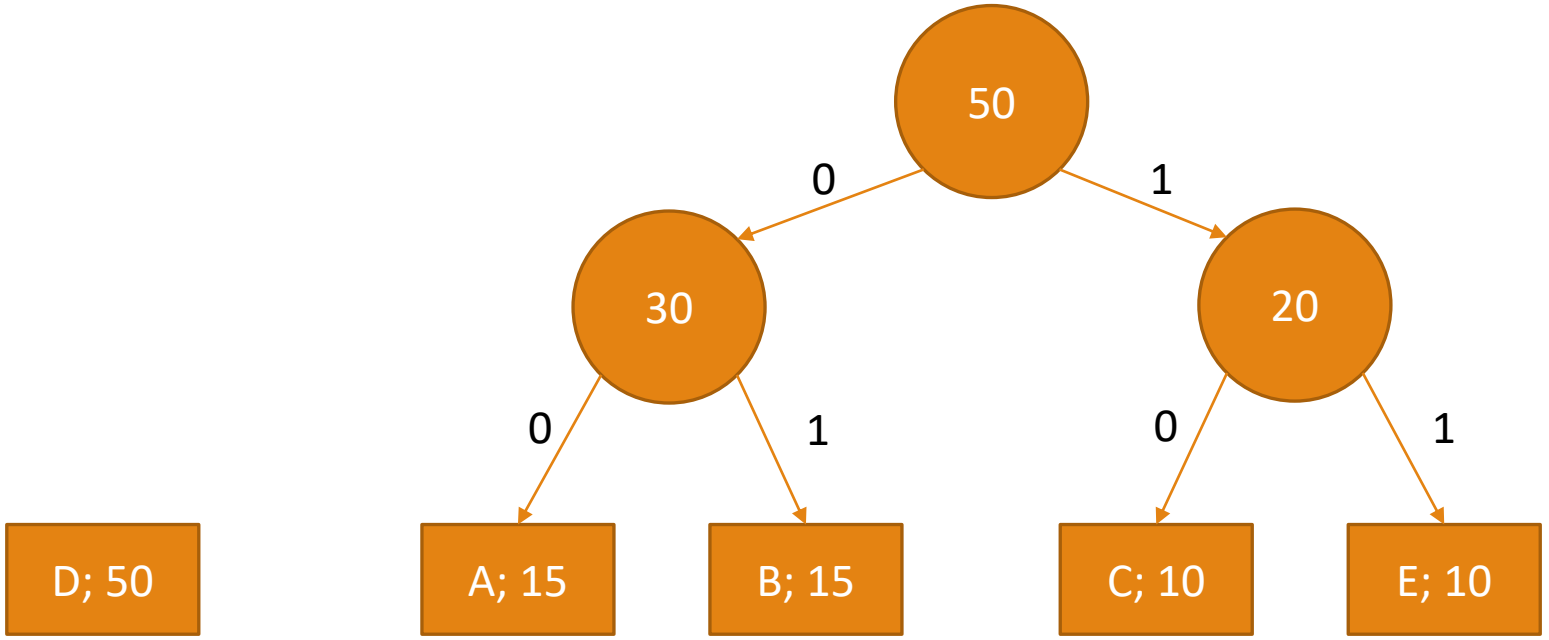
Exercise 3: Huffman coding



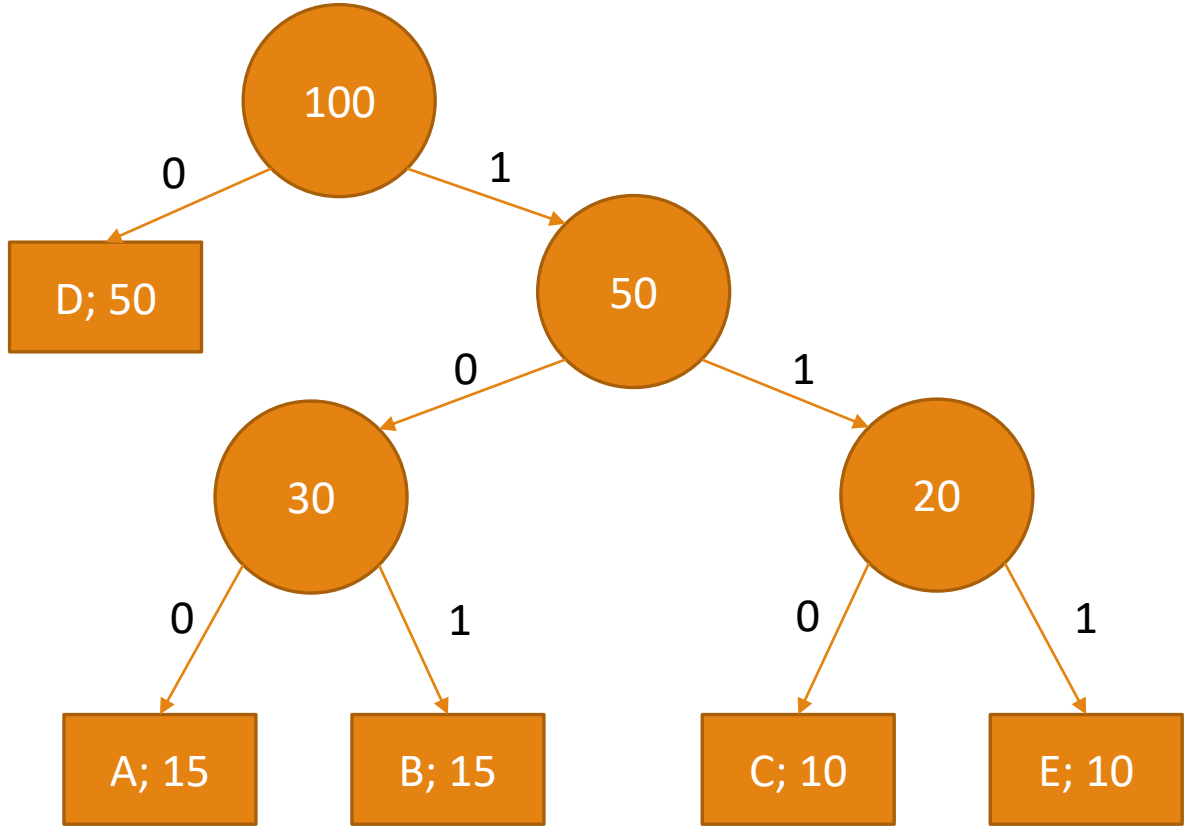
Exercise 3: Huffman coding



Exercise 3: Huffman coding



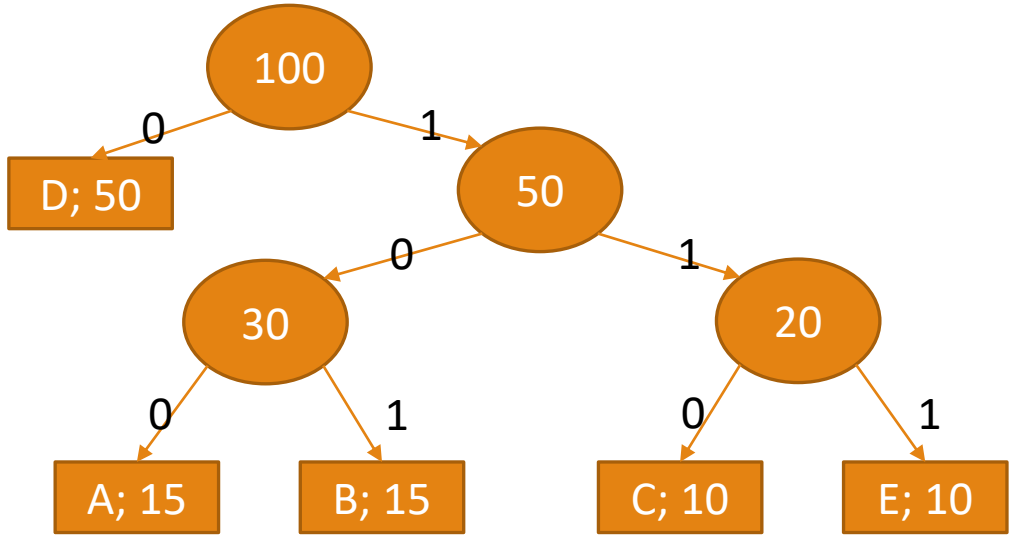
Exercise 3: Huffman coding



Exercise 3: Huffman coding

1. What is the encoding of **dba** in the tree?

2. What is the decoding of the string 1010110100?



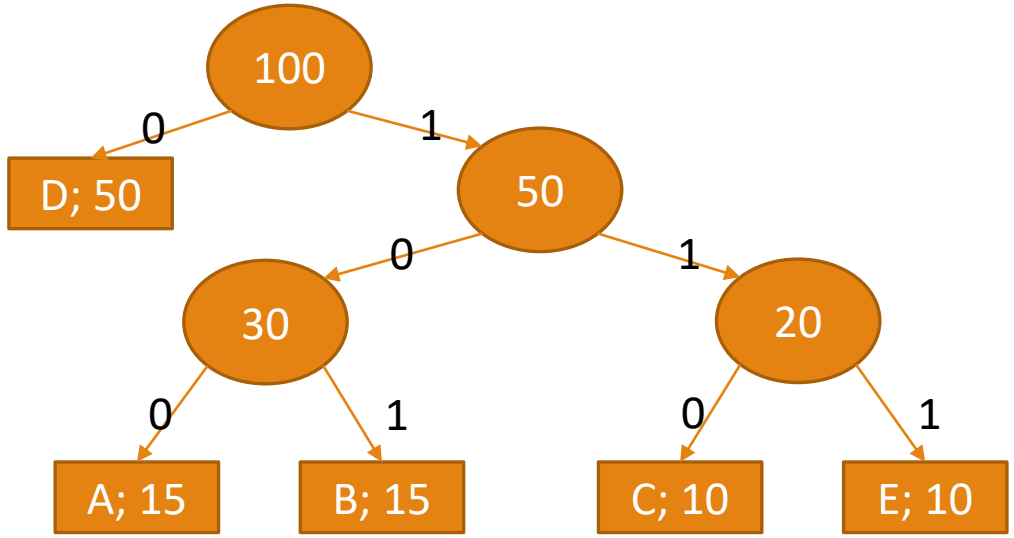
Exercise 3: Huffman coding

1. What is the encoding of **dba** in the tree?

0101100

2. What is the decoding of the string 1010110100?

bdca



Exercise 4: More Huffman Coding (Fin22B)

1. Huffman algorithm is performed on a text over an alphabet {a,b,c,d,e,f}. It is known that $p(a) \geq p(b) \geq p(c) \geq p(d) \geq p(e)$.

The table presents partial information regarding the input and the output of the algorithm. Complete the missing entries in the table. Specifically, you should determine frequencies of b,d,e, and f, and optimal code-words for a, c, and e, such that the table presents a valid input and output of the algorithm.

Letter	a	b	c	d	e	f
Frequency	0.6		0.1			
code		100		110		1111

Exercise 4: More Huffman Coding

a appears in the **majority** of the time, so it must have a 1-letter code.

b starts with 1, therefore **a** must be 0.

Letter	a	b	c	d	e	f
Frequency	0.6		0.1			
code	0	100		110		1111

Exercise 4: More Huffman Coding

We have no potential remaining 2-letter codes. We only have one remaining 3 letter code.

Since it has frequency 0.1, we know **c** has more appearances than at least one of **b,d,e**.

We also need at least one 4 letter code in addition to **f.**

Therefore **e** must have a 4 letter code, which must be 1110, and therefore **c** has a 3 letter code – which needs to be 101 so we remain prefix free.

Letter	a	b	c	d	e	f
Frequency	0.6		0.1			
code	0	100	101	110	1110	1111

Exercise 4: More Huffman Coding

Lastly, we just need to choose frequencies that match the constraint $p(a) \geq p(b) \geq p(c) \geq p(d) \geq p(e)$ and the chosen codes.

Letter	a	b	c	d	e	f
Frequency	0.6	0.15	0.1	0.1	0.03	0.02
code	0	100	101	110	1110	1111

Exercise 5: Even More Huffman Coding

For each of the four candidate codes over the alphabet {A,B,C,D,E}, decide whether it *can* be produced by Huffman's algorithm for some set of positive frequencies

If **YES** - give one valid frequency vector.

If **NO** - give a short, precise explanation why not.

(i) A=0, B=100, C=101, D=110, E=111

(ii) A=00, B=01, C=10, D=110, E=111

(iii) A=0, B=01, C=10, D=110, E=111

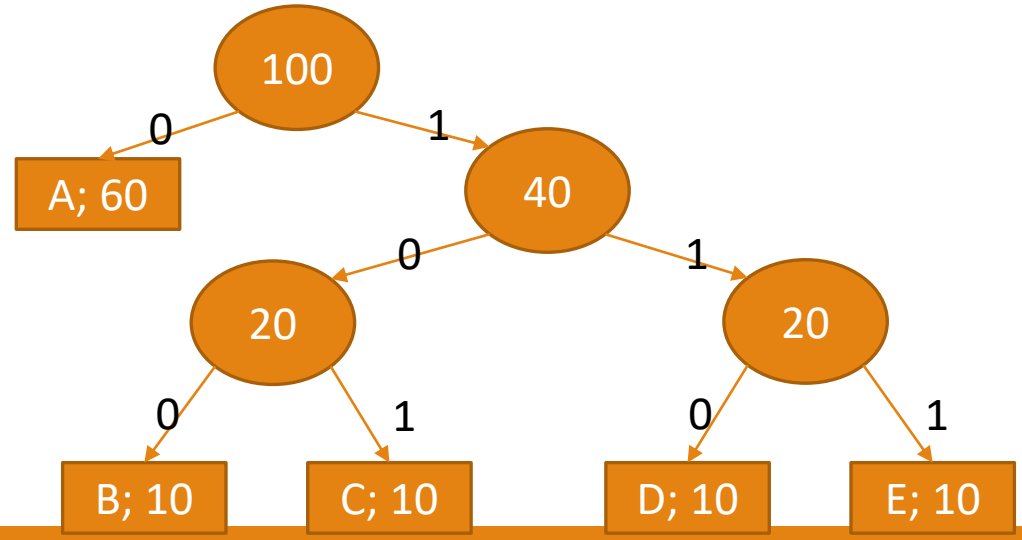
(iv) A=0, B=100, C=110, D=1110, E=1111

Exercise 5: Even More Huffman Coding

(i) A=0, B=100, C=101, D=110, E=111.

Yes.

p(A)=60, p(B)=10, p(C)=10, p(D)=10, p(E)=10

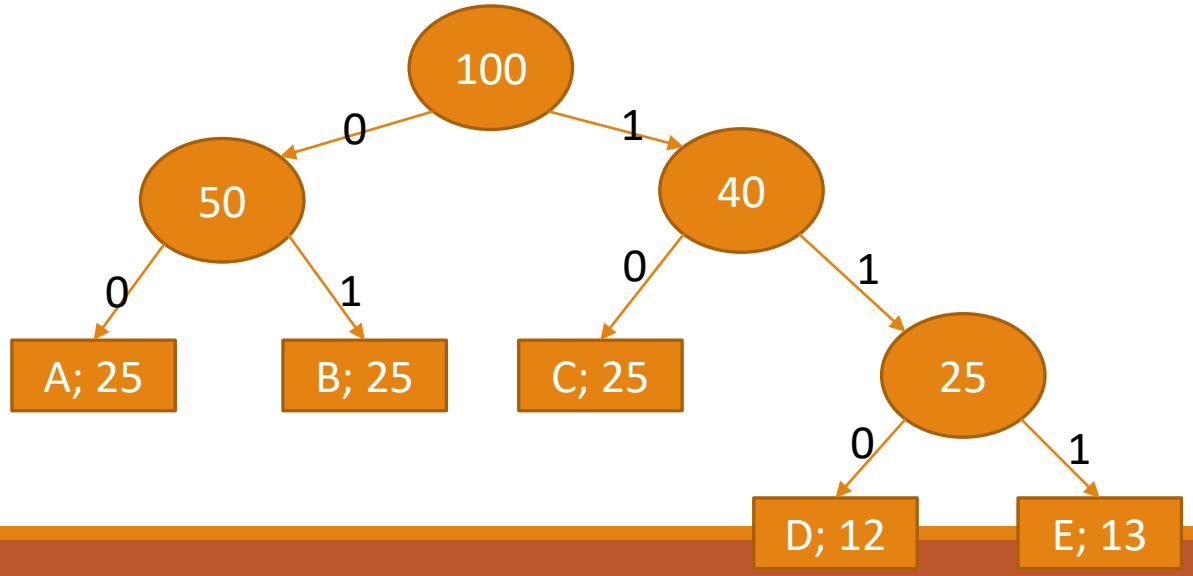


Exercise 5: Even More Huffman Coding

(ii) A=00, B=01, C=10, D=110, E=111

Yes.

p(A)=25, p(B)=25, p(C)=25, p(D)=12, p(E)=13



Exercise 5: Even More Huffman Coding

(iii) A=0, B=01, C=10, D=110, E=111.

No: Because A=0 is a prefix of B=01.

A Huffman code must be prefix-free

Exercise 5: Even More Huffman Coding

(iv) A=0, B=100, C=110, D=1110, E=1111.

No: This code is prefix free, but the codeword for B can be shortened to B=10.

Exercise 6: Filling gas

A driver needs to travel a long journey. Along the way, there are n gas stations. Denote by d_i the distance of the i-th station from the starting point. After stopping at a gas station, the fuel tank is full. When the fuel tank is full, the car has enough fuel to drive m kilometers.

At the beginning of the journey, the fuel tank is full. It is known that $d_1 \leq m$, and that

$d_{i+1} - d_i \leq m$, for every $1 \leq i < n$. Also, the distance from the last gas station to the final destination is at most m.

Suggest an algorithm for determining the gas stations in which the driver should stop, so that her number of stops along the journey will be minimal.

1. Describe the algorithm and analyze its time complexity.

2. State the corresponding 'greedy choice property'. Claim and prove it.

Exercise 6: Algorithm

Assume that the stations are sorted such that $d_1 \leq d_2 \leq \cdots \leq d_n$.

1. Initially $Location = 0$.

2. If we are more than m km away from the final destination:

3. Let k be the **maximal** index of a gas station such that $d_k - Location \leq m$.

4. Drive to station k and fill the tank there.

5. Update $Location = d_k$.

6. Go to step 2.

Another formulation: Whenever we reach a gas station, if we can still make it to the next station, do not fill the tank. Formally: after filling at location d_j (or 0), when arriving to a station at distance d_i, fill the tank only if $d_{i+1} - d_j > m$.

Exercise 6: GCP

Greedy choice property: there exists an optimal solution in which the first stop is at station k such that $d_k \leq m$ and $d_{k+1} > m$.

Proof: Let A be a solution in which the first stop is at station $k' \neq k$. It must be that $k' < k$ as otherwise, it is impossible to reach station k'. Since $d_k \leq m$, it is possible to delay the first stop to station k. Since station k is after station k', the remainder of the trip can be as in A.

Therefore, A can be changed so that the first stop is at station k without increasing the number of stops.